CSC-239
Java Programming:
Chapter 3: Selections

September 10, 2025

NOTICE to Students: Portions of these lecture slides are
Copyright 2021 Pearson Education, Inc.
We have a license to use this material ***only*** in the context of this course.
There is NO PERMISSION to share these lecture slides with anyone not taking the course.
There is NO PERMISSION to post these lecture slides on the Internet.

Reminder

- Quiz 1 will begin on Monday, September 15.
  - Online (Moodle) Quiz.
  - The Quiz will open Monday evening, at 9:00 p.m.
  - The Quiz will remain open for approximately one week.
  - Each student will have two hours to complete the Quiz, from the time that they begin.
- The material covered on Quiz 1 will be
  - The content of the course Syllabus
  - The lectures of September 3 and 8.
  - The quiz will include True / False, and Multiple-Choice questions.

Online Information

- Java Tutorials:

  https://docs.oracle.com/javase/tutorial/

- Java API:

  https://docs.oracle.com/en/java/javase/21/docs/api/index.html

- The official "*Grader Than*" documentation:

  https://docs.graderthan.com/student/

- *Grader Than* Support E-mail:

  support@graderthan.com

Comparison Operators

Java Operator	Mathematics Symbol	Name	Example (radius is 5)	Result
<	<	less than	`radius < 0`	`false`
<=	≤	less than or equal to	`radius <= 0`	`false`
>	>	greater than	`radius > 0`	`true`
>=	≥	greater than or equal to	`radius >= 0`	`true`
==	=	equal to	`radius == 0`	`false`
!=	≠	not equal to	`radius != 0`	`true`

The `boolean` Type and Operators

- The result of the comparison is a Boolean value: true or false.

- This result can be used right away (in a decision statement):

```
if(radius > 0) {    . . .    }
```

- The Boolean value can be remembered, if necessary, in a variable of the `boolean` data type:

```
boolean isValid = (radius > 0);
```

One-way `if` Statements

```
if (boolean-expression) {
    statement(s);
}
```

- The curly braces are necessary only if more than one statement is present

FOR EXAMPLE:

```
if (radius >= 0) {
    area = radius * radius * PI;
    System.out.println("The area"
    + " for the circle of radius "
    + radius + " is " + area);
}
```

The Two-way `if` Statement

```
if (boolean-expression) {
      statement(s)-for-the-true-case;
}
else {
      statement(s)-for-the-false-case;
}
```

`if-else` Example

```
if (radius >= 0) {
  area = radius * radius * 3.14159;

  System.out.println("The area for a "
    + "circle of radius " + radius +
    " is " + area);
}
else {
  System.out.println("Negative input");
}
```

Multiple Alternative if Statements

```
if (score >= 90.0)
  grade = 'A';
else
  if (score >= 80.0)
    grade = 'B';
  else
    if (score >= 70.0)
      grade = 'C';
    else
      if (score >= 60.0)
        grade = 'D';
      else
        grade = 'F';
```

Equivalent

```
if (score >= 90.0)
  grade = 'A';
else if (score >= 80.0)
  grade = 'B';
else if (score >= 70.0)
  grade = 'C';
else if (score >= 60.0)
  grade = 'D';
else
  grade = 'F';
```

What is Wrong With This Code? (1)

```
if (radius >= 0);
{
  area = radius*radius*PI;
  System.out.println(
    "The area for the circle of radius " +
    radius + " is " + area);
}
```

What is Wrong With This Code? (2)

```
if (radius >= 0);      //  Wrong!
{
  area = radius*radius*PI;
  System.out.println(
    "The area for the circle of radius " +
    radius + " is " + area);
}
```

- Adding a semicolon at the end of an if clause is a common mistake.
- This mistake is hard to find, because it is not a compilation error or a runtime error, it is a logic error.
- This error often occurs when you use the next-line block style.

What is Wrong With This Code? (3)

- The erroneous code (in the previous two slides) is equivalent to the following:

```
if (radius >= 0)
{
}
{
  area = radius*radius*PI;
  System.out.println(
    "The area for the circle of radius " +
    radius + " is " + area);
}
```

Switch Statement

```
switch (switch-expression) {
    case value1:    statement(s);
                    break;
    case value2:    statement(s);
                    break;
    …
    case valueN:    statements(s);
                    break;
    default:        statements(s);
}
```

Logical Operators

Operator	Name	Evaluates to TRUE if
!	NOT	input is FALSE
&&	AND	both inputs are TRUE
||	OR	either input is TRUE
^	Exclusive OR	one input is TRUE (but not both)

Logic Operations: NOT

- One input, one output.
- The output is TRUE if the input is FALSE.
- Also called “negation” or an “inverter”.
- We can represent this with a “truth table” (assume 0 indicates FALSE and 1 indicates TRUE):

NOT	0	1
	1	0

Logic Operations: AND

- Two inputs, one output
- The output is TRUE if *both* inputs are TRUE

AND	0	1
0	0	0
1	0	1

Logic Operations: OR

- Two inputs, one output
- The output is TRUE if *either* input is TRUE

OR	0	1
0	0	1
1	1	1

Logic Operations: Exclusive-OR

- Two inputs, one output
- The output is TRUE if *either* input is TRUE, but not if *both* are TRUE
- Sometimes abbreviated as "XOR"

XOR	0	1
0	0	1
1	1	0

TestBooleanOperators.java (1)

```
// TestBooleanOperators
// (see also Listing 3.6, page 94)

import java.util.Scanner;

public class TestBooleanOperators {
  public static void main(String[] args) {
    // Create a Scanner
    Scanner input = new Scanner(System.in);

    // Receive an input
    System.out.print("Enter an integer: ");
    int number = input.nextInt();
```

TestBooleanOperators.java (2)

```
    if (number % 2 == 0 && number % 3 == 0)
      System.out.println(number +
         " is divisible by 2 and 3.");

    if (number % 2 == 0 || number % 3 == 0)
      System.out.println(number +
          " is divisible by 2 or 3.");

    if (number % 2 == 0 ^ number % 3 == 0)
      System.out.println(number +
      " is divisible by 2 or 3, but not both.");
  } // (end main)
} // (end class TestBooleanOperators)
```

Problem: Determining Leap Year

- In every day experience, we think of a *Leap Year* as being a year which is a multiple of 4. However, there is an exception to this rule:
  - A year which is a multiple of 100 is *not* a leap year, *unless* it is also a multiple of 400.
- In other words, a leap year is a year which:
  - is divisible by 4 but not divisible by 100, OR
  - is divisible by 400.

```
int y;
boolean isLeapYear;
isLeapYear = (y%4 == 0 && y%100 != 0)
                    || (y%400==0);
```

Conditional Operator

Ternary operator:

(*boolean-expression*) ? *expression1* : *expression2*

```
y = (x > 0) ? 1 : -1;
```

is equivalent to

```
if (x > 0)
        y = 1;
else
        y = -1;
```

- Ternary operator: 3 operands
- Binary operator: 2 operands
- Unary operator: 1 operand

Conditional Operator Example

```
System.out.println(
   (num % 2 == 0)? num + "is even" :
   num + "is odd");
```

- The sample code above does the same thing as the sample code below:

```
if (num % 2 == 0)
   System.out.println(num + "is even");
else
   System.out.println(num + "is odd");
```

Formatting Output

Use the printf method.

`System.out.printf(`*format*`, `*items*`);`

Where *format* is a string consisting of

- literal text
- A *format specifier* for each *item* in the list.

  A *format specifier* specifies how an item should be displayed.

Frequently-Used Format Specifiers

Specifier	Output	Example
`%b`	a boolean value	`true or false`
`%c`	a character	`'a'`
`%d`	a decimal integer	`200`
`%f`	a floating-point number	`45.460000`
`%e`	a number in scientific notation	`4.556000e+01`
`%s`	a string	`"Java is cool"`

`printf` Example

```
int count = 5;
double amount = 45.56;
System.out.printf(
    "count is %d, amount is %f",
    count, amount);
```

Resulting output:

```
count is 5, amount is 45.560000
```

Example Format Specifier

- Suppose you wish to round-off the fractional part of a number to a particular number of decimal places, such as when displaying currency:
- An example format for displaying two decimal places is:

  ```
  %7.2f
  ```

  where the minimum number of characters written to the output is 7,

  and the number of decimal places is 2.

Remember This Slide (Lecture01)?

- Calculations involving floating-point numbers are approximated because these numbers are not stored with complete accuracy. For example,

```
System.out.println(1.0 - 0.1 - 0.1 - 0.1 - 0.1 - 0.1);
```

displays `0.5000000000000001`, not `0.5`, and

```
System.out.println(1.0 - 0.9);
```

displays `0.09999999999999998`, not `0.1`.

- Integers are stored precisely. Therefore, calculations with integers yield a precise integer result.

PrintfDemo.java

- This program experiments with a few floating point calculations, and how to use the System.out.printf() method to control the output.
- This program also contains a deliberate "endless loop":
  - Is more convenient when we are just tinkering with different input values.
  - Provides an opportunity to show you how to "kill" a runaway program using NetBeans.

Operator Precedence

- Determines which operations are performed first in an expression.
- Common binary operations are consistent with rules of Algebra:

```
3 + 2 * 7
```

evauates to 17 (multiplication before addition)

- Other operations are less intuitive
- Consult Appendix C

Operator Precedence (continued)

- Online Reference:

http://docs.oracle.com/javase/tutorial/java/nutsandbolts/operators.html

- When in doubt, use parentheses to be sure the compiler understands your intentions.
- Parentheses can also help clarify the code for a human reader.

Operator Associativity

- When two operators with the same precedence are evaluated, the *associativity* of the operators determines the order of evaluation.
- All binary operators except assignment operators are *left-associative*.

```
a - b + c - d
```

is equivalent to

```
((a - b) + c) - d
```

- Assignment operators are *right-associative*.

```
a = b += c = 5
```

is equivalent to

```
a = (b += (c = 5))
```

Summary

- Comparison Operators, `boolean` data type.
- `if` statements, `if-else` statements.
- `switch` statement
- Logical Operators
- Conditional Operator
- The `printf` method for formatting Output.
- Operator Precedence and Associativity

Lab03a: Days in a Month

- In this Lab Exercise, you will write a program that inputs a numeric month value, and a particular year.
- The program will then state how many days are contained in that month.